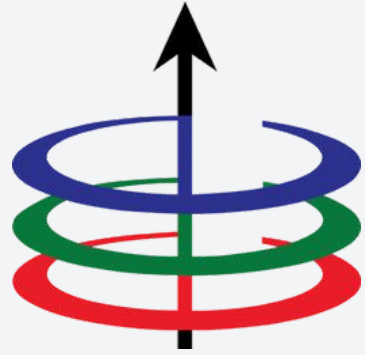




Lecture 2.3

Sorting Algorithms I

EEE 121 THVW1

Steven S. Sison

- Sorting
- Sorting Algorithms 1: Insertion Sort
- Sorting Algorithms 2: Bubble Sort
- Sorting Algorithms 3: Selection Sort

- **Sorting**
- Sorting Algorithms 1: Insertion Sort
- Sorting Algorithms 2: Bubble Sort
- Sorting Algorithms 3: Selection Sort

What is Sorting?

Sorting is the process of arranging an object in order.

- For example, we have an array of integers.

$$(a_0, a_1, \dots, a_n)$$

- Sorting is the process of arranging them into either ascending or descending order such that:

$$a_0 \leq a_1 \leq \dots \leq a_n \text{ for ascending order}$$

$$a_0 \geq a_1 \geq \dots \geq a_n \text{ for descending order}$$

Example:

Input

4	7	1	5	2	8	3	6
---	---	---	---	---	---	---	---



Output

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Sorting is very useful in programming

We use sorting in almost every basic tasks:

- **Searching the web** - when you search a certain webpage in google, google ranks it through their proprietary algorithms and then sorts it based on its ranking

Sorting is very useful in the real world

We use sorting in almost every basic tasks:

- **Searching the web** - when you search a certain webpage in google, google ranks it through their proprietary algorithms and then sorts it based on its ranking
- **Shopee, Lazada, and E-commerce Sites** - whenever you shop at these sites, there is often an option to sort it either by price, or popularity. With the vast number of products out there, sorting algorithms must be efficient to display results quickly.

What are common sorting algorithms?

You are not expected to create your own sorting algorithm. Throughout history, computer scientists have already developed efficient algorithms for sorting. We'll explore some of them today:

- Insertion Sort
- Bubble Sort
- Selection Sort
- Merge Sort
- Quick Sort

What are common sorting algorithms?

You are not expected to create your own sorting algorithm. Throughout history, computer scientists have already developed efficient algorithms for sorting. We'll explore some of them today:

- Insertion Sort
- Bubble Sort
- Selection Sort
- Merge Sort
- Quick Sort

What are common sorting algorithms?

You are not expected to create your own sorting algorithm. Throughout history, computer scientists have already developed efficient algorithms for sorting. We'll explore some of them today:

- Insertion Sort
- Bubble Sort
- Selection Sort
- Merge Sort
- Quick Sort

To get you started, you can visit this [site](#).

We'll fill up this table as we go through the algos

In this lecture, we want to practice our algorithm analysis techniques to prove and determine the time complexities of sorting algorithms.

Algorithm	Proved?	Time Complexity	
		Best Case	Worst Case
Insertion Sort			
Bubble Sort			
Selection Sort			
Merge Sort			
Quick Sort			

- Sorting
- **Sorting Algorithms 1: Insertion Sort**
- Sorting Algorithms 2: Bubble Sort
- Sorting Algorithms 3: Selection Sort

Insertion Sort

Insertion sort works by dividing the array into two: sorted and unsorted. For each element in the unsorted array, extract the element. Iterate over the sorted array, and insert the extracted element into its proper location. **Keyword: Insert**

6 5 3 1 8 7 2 4

Pseudocode:

```
mark first element as sorted
```

```
for each unsorted element X
```

```
    'extract' the element X
```

```
    for j = lastSortedIndex down to 0
```

```
        if current element j > X
```

```
            move sorted element to the right by 1
```

```
        break loop and insert X here
```

Insertion Sort - Implementation

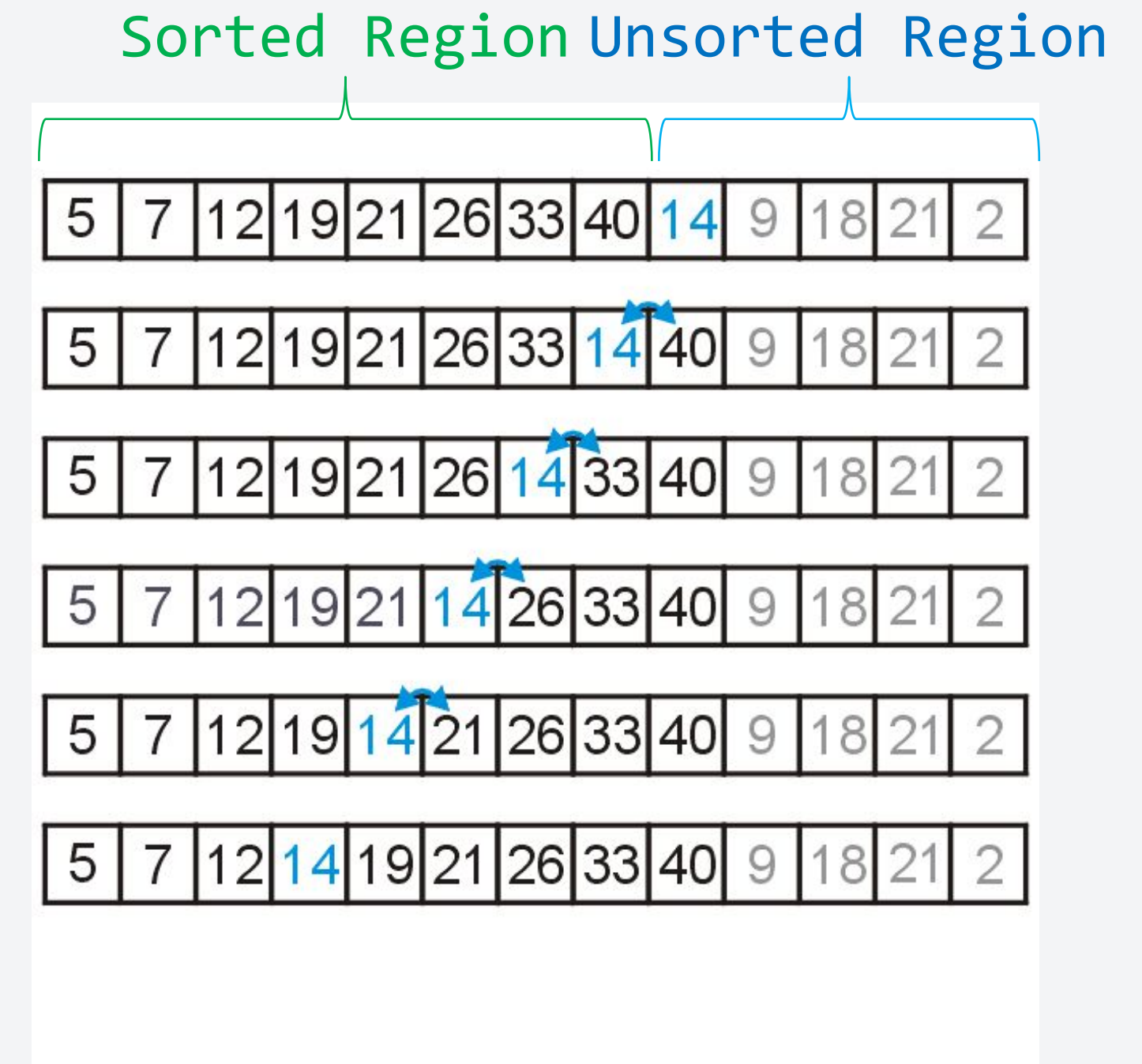
Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {
    for(int i = 1; i < n; i++) {
        int j = i - 1;
        while(j >= 0 && array[j] > array[j+1]) {
            std::swap(array[j], array[j+1]);
            //std::swap is from standard
            // algorithm library
            // #include <algorithm>

            j--;
        }
    }
}
```

Two regions: Sorted and unsorted.

For each iteration, extract one from unsorted region then insert it to its proper location in sorted region.



Demo: Insertion Sort

Insertion Sort - Implementation

We know how insertion sort works. But the question remains, does it actually work?

A

4	3	1	5	2
---	---	---	---	---

:
Move **A[1]** leftwards until you find something smaller (or can't go move it any further).

B

3	4	1	5	2
---	---	---	---	---

:
Do the same thing for **A[2]**.

C

1	3	4	5	2
---	---	---	---	---

:
And also for **A[3]** (it's already in the right position).

D

1	3	4	5	2
---	---	---	---	---

:
And lastly for **A[4]**.

E

1	2	3	4	5
---	---	---	---	---

:
Then we're done!

However, showing that it works for one case is not enough proof that it's correct!

Insertion Sort - Implementation

We know how insertion sort works. But the question remains, does it actually work?

4	3	1	5	2
---	---	---	---	---

3	4	1	5	2
---	---	---	---	---

The first element, [4], is a sorted list. 3 is our other element.

Correctly inserting 3 into the sorted list [4] produces another sorted list [3, 4] that's longer by one element.

3	4	1	5	2
---	---	---	---	---

1	3	4	5	2
---	---	---	---	---

The first two elements, [3, 4], are a sorted list. 1 is our other element.

Correctly inserting 1 into the sorted list [3, 4] produces another sorted list [1, 3, 4] that's longer by one element.

1	3	4	5	2
---	---	---	---	---

1	3	4	5	2
---	---	---	---	---

The first three elements, [1, 3, 4], are a sorted list. 5 is our other element.

Correctly inserting 5 into the sorted list [1, 3, 4] produces another sorted list [1, 3, 4, 5] that's longer by one element.

1	3	4	5	2
---	---	---	---	---

The first four elements, [1, 3, 4, 5], are a sorted list. 2 is our other element.

Correctly inserting 2 into the sorted list [1, 3, 4, 5] produces another sorted list [1, 2, 3, 4, 5] that's longer by one element.

Insertion Sort - Let's prove it!

Try natin i-prove informally. By intuition kumbaga (tiwala sa utak),

- Suppose you have a sorted list,

1	3	4	5
---	---	---	---

, and another element

2

- Inserting

2

 immediately to the right of the largest element from the original list that's smaller than

2

 (i.e. right of

1

) produces another sorted list. Notice that this new list is longer than the original one by one element:

1	2	3	4	5
---	---	---	---	---

.
- Starting from a sorted list of length 1 and unsorted list of element $n - 1$, we are able to place all n elements in sorted region at algorithm termination.

Insertion Sort - Let's prove it!

Now let's try proving it formally.

Given the sample implementation that we have earlier, what do you think is the best technique to use?

Insertion Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

Any idea on the loop invariant?

Insertion Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {
    for(int i = 1; i < n; i++) {
        int j = i - 1;
        while(j >= 0 && array[j] > array[j+1]) {
            std::swap(array[j],array[j+1]);
            //std::swap is from standard
            // algorithm library
            // #include <algorithm>

            j--;
        }
    }
}
```

Loop Invariant: array[:i+1] is sorted

Insertion Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

Base Case: At $i = 0$, `array[:1]` contains only one element. Thus, it is sorted.

Loop Invariant: `array[:i+1]` is sorted

Insertion Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

Loop Invariant: array[:i+1] is sorted

Base Case: At $i = 0$, array[:1] contains only one element. Thus, it is sorted.

Maintenance: At iteration i , the loop is sorted. Thus, array[: i] is sorted with i elements. At the next iteration, correctly inserting the next element produces a sorted array that is longer by one element. Thus, array[: $i+1$] is sorted.

3	4	1	5	2
1	3	4	5	2

Inserting 1 in its correct place increases the size of the array by 1!

Insertion Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

Loop Invariant: array[:i+1] is sorted

Base Case: At $i = 0$, array[:1] contains only one element. Thus, it is sorted.

Maintenance: At iteration i , the loop is sorted. Thus, array[: i] is sorted with i elements. At the next iteration, correctly inserting the next element produces a sorted array that is longer by one element. Thus, array[: $i+1$] is sorted.

Conclusion: At termination ($n-1$), array[: n] is sorted. Since the array only has n elements, the whole list is sorted. Done!

Insertion Sort - What about the runtime?

Now let's determine its worst time complexity,

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

Insertion Sort - What about the runtime?

Now let's determine its worst time complexity,

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

At most n inner
iterations, $O(n)$

Insertion Sort - What about the runtime?

Now let's determine its worst time complexity,

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

At most n inner
iterations, $O(n)$

At most n outer
iterations, $O(n)$

Insertion Sort - What about the runtime?

Now let's determine its worst time complexity,

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
            j--;  
        }  
    }  
}
```

At most n inner
iterations, $O(n)$

At most n outer
iterations, $O(n)$

$$T(n) = O(n)O(n) = O(n^2)$$

Insertion Sort - What about the runtime?

Sample C++ Implementation:

```
void insertion_sort(int* array, const int n ) {  
    for(int i = 1; i < n; i++) {  
        int j = i - 1;  
        while(j >= 0 && array[j] > array[j+1]) {  
            std::swap(array[j],array[j+1]);  
            //std::swap is from standard  
            // algorithm library  
            // #include <algorithm>  
  
            j--;  
        }  
    }  
}
```

What about the best case?

- The best case is when the input array is already sorted.
- If the array is already sorted, the while loop will no longer execute since $\text{array}[j+1] > \text{array}[j]$.

$$T(n) = O(n)$$

- Sorting
- Sorting Algorithms 1: Insertion Sort
- **Sorting Algorithms 2: Bubble Sort**
- Sorting Algorithms 3: Selection Sort

Bubble Sort

Bubble sort works by finding the largest element in the array and **bubble it** out to the last index of the array.

6 5 3 1 8 7 2 4

Pseudocode:

```
do
  swapped = false
  for i = 1 to indexOfLastUnsortedElement-1
    if leftElement > rightElement
      swap(leftElement, rightElement)
      swapped = true; ++swapCounter
  while swapped
```

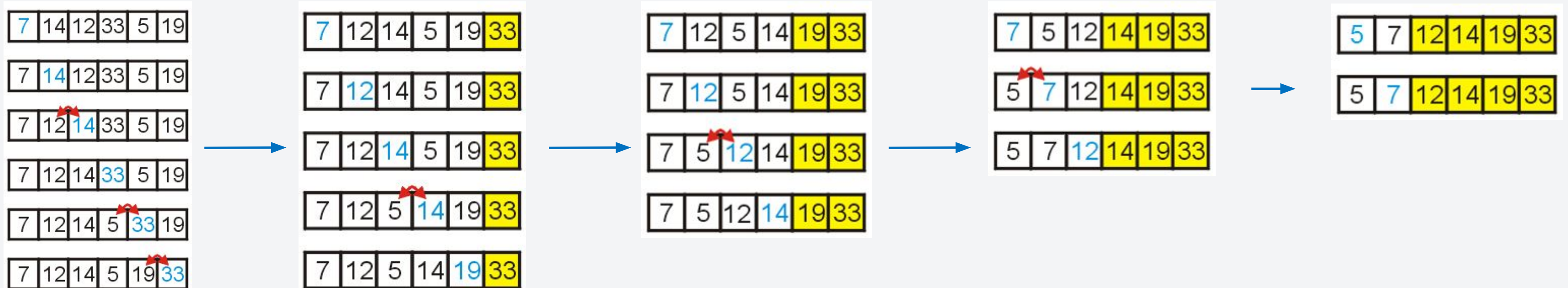
Iterate over each element in the array. If the left element is greater than the right, swap. Repeat until end of array.

Bubble Sort

Sample C++ Implementation:

You can easily prove its correctness and show that it has $O(n^2)$ time complexity

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```



Demo: Bubble Sort

Bubble Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

Inner Loop Invariant: array[i]
contains the largest element

Outer Loop Invariant: the last n-i
elements are sorted

Base Case:

- **Outer LI:** At $i = n-1$, array[n-1:n] contains only one element. Thus, it is sorted.
- **Inner LI:** At $j = 0$, the larger element is at array[j+1] due to the swap.

Bubble Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

Inner Loop Invariant: array[j+1]
contains the largest element

Outer Loop Invariant: the last n-i
elements are sorted

Maintenance:

- **Inner LI:** At iteration j, the swapping of elements ensure that the larger element is at array[j+1].
- **Outer LI:** At iteration i, the inner loop ensures that the largest element in the **unsorted array** is at array[i]. Thus, the invariant that the last n-i elements are sorted is still true.

Bubble Sort - Let's prove it!

Now let's try proving it formally. Let's use **Loop Invariants**!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

Inner Loop Invariant: array[i]
contains the largest element

Outer Loop Invariant: the last n-i
elements are sorted

Termination:

- **Inner LI:** At $j = i-1$, the largest element is put to $\text{array}[j+1] = \text{array}[i]$. Thus, the invariant holds true. Done!
- **Outer LI:** At $i = 1$, only one element is not sorted. Since the $n-1$ elements are already sorted, it means that $\text{array}[0:n]$ is sorted. Done!

Bubble Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

Bubble Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

} At most n inner iterations, $O(n)$

Bubble Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

At most n inner
iterations, $O(n)$

At most n outer
iterations, $O(n)$

Bubble Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

At most n inner
iterations, $O(n)$

At most n outer
iterations, $O(n)$

$$T(n) = O(n)O(n) = O(n^2)$$

Bubble Sort - Gano kabilis?

What about the best case?

Sample C++ Implementation:

```
void bubble(int* array, const int n ) {  
    for ( int i = n - 1; i > 0; --i ) {  
        for ( int j = 0; j < i; ++j ) {  
            if ( array[j] > array[j + 1] )  
                std::swap( array[j], array[j + 1] );  
        }  
    }  
}
```

Similar to insertion sort, best case happens when the array is already sorted.

- In the given code, if the array is already sorted, we would still have to pass through over the whole array, $T(n) = O(n^2)$

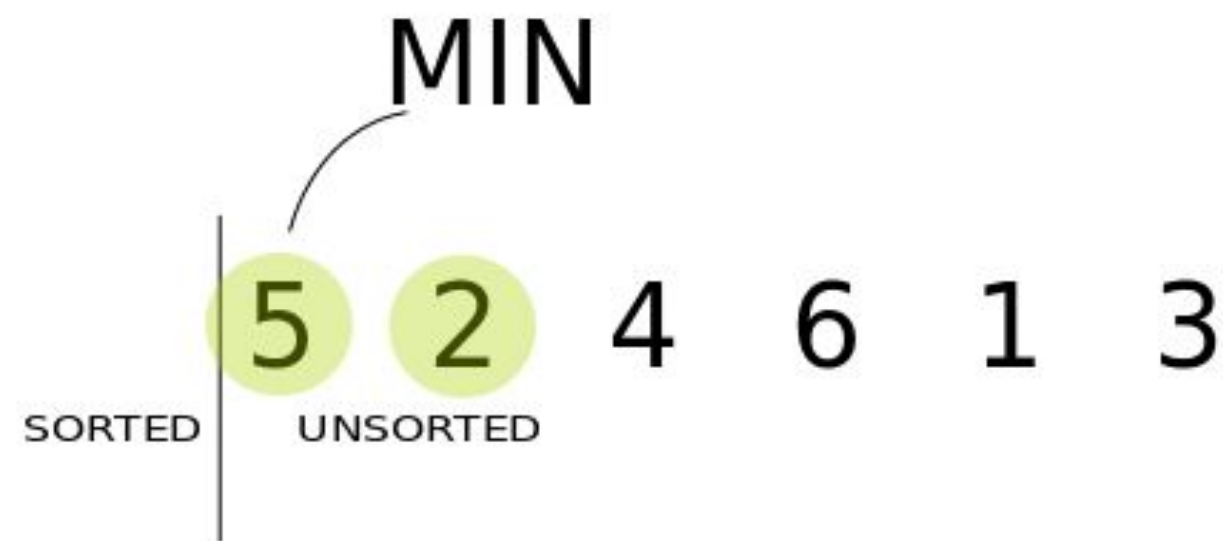
However, we can modify the code to include a swap counter. If the swap counter is 0, it means the array is already sorted. We can stop.

$$T(n) = O(n)$$

- Sorting
- Sorting Algorithms 1: Insertion Sort
- Sorting Algorithms 2: Bubble Sort
- **Sorting Algorithms 3: Selection Sort**

Selection Sort

Selection sort works by dividing the array into sorted and unsorted. The algorithm works by finding the smallest element in the unsorted array and then swapping it at the first element of the unsorted array. **Keyword: Selecting**



Pseudocode:

```
repeat (numOfElements - 1) times
    set the first unsorted element as the minimum
    for each of the unsorted elements
        if element < currentMinimum
            set element as new minimum
    swap minimum with first unsorted position
```

Selection Sort

Intuition:

- Scan all items and find the smallest in **unsorted region**
- Swap into position as the first item of the **unsorted region**
- Repeat on the remaining N-1 times

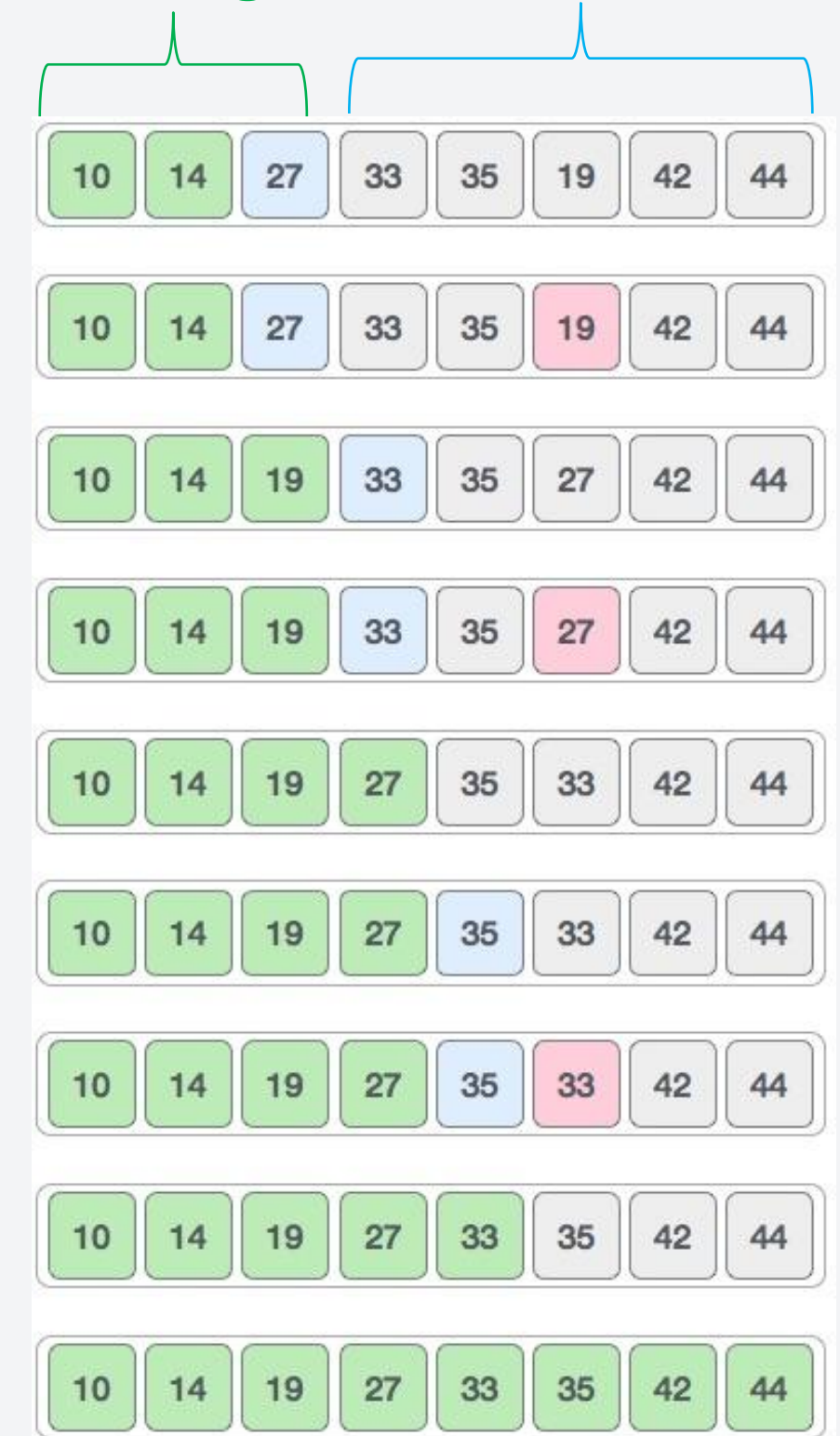
Sample C++

Implementation:

Proving is left as an assignment :)

```
void selection(int* array, const int n ) {  
    int min;  
    for(i = 0; i < n; i++) {  
        min = i;  
        for(j = i+1; j < n; j++) {  
            if(array[j] < array[min])  
                min = j;  
        }  
        if(min != i)  
            std::swap(array[i], array[min]);  
    }  
}
```

Sorted Region Unsorted Region



Demo: Selection Sort

Selection Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void selection(int* array, const int n ) {  
    int min;  
    for(i = 0; i < n; i++) {  
        min = i;  
        for(j = i+1; j < n; j++) {  
            if(array[j] < array[min])  
                min = j;  
        }  
        if(min != i){  
            std::swap(array[i],array[min]);  
        }  
    }  
}
```

Selection Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void selection(int* array, const int n ) {  
    int min;  
    for(i = 0; i < n; i++) {  
        min = i;  
        for(j = i+1; j < n; j++) {  
            if(array[j] < array[min])  
                min = j;  
        }  
        if(min != i){  
            std::swap(array[i],array[min]);  
        }  
    }  
}
```

} At most n inner
iterations, $O(n)$

Selection Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void selection(int* array, const int n ) {  
    int min;  
    for(i = 0; i < n; i++) {  
        min = i;  
        for(j = i+1; j < n; j++) {  
            if(array[j] < array[min])  
                min = j;  
        }  
        if(min != i){  
            std::swap(array[i],array[min]);  
        }  
    }  
}
```

} At most n inner
iterations, $O(n)$

} At most n outer
iterations, $O(n)$

Selection Sort - Gano kabilis?

Let's solve for the worst time complexity!

Sample C++ Implementation:

```
void selection(int* array, const int n ) {  
    int min;  
    for(i = 0; i < n; i++) {  
        min = i;  
        for(j = i+1; j < n; j++) {  
            if(array[j] < array[min])  
                min = j;  
        }  
        if(min != i){  
            std::swap(array[i],array[min]);  
        }  
    }  
}
```

} At most n inner
iterations, $O(n)$

} At most n outer
iterations, $O(n)$

$$T(n) = O(n)O(n) = O(n^2)$$

Selection Sort - Gano kabilis?

What about the best case?

Sample C++ Implementation:

```
void selection(int* array, const int n ) {
    int min;
    for(i = 0; i < n; i++) {
        min = i;
        for(j = i+1; j < n; j++) {
            if(array[j] < array[min])
                min = j;
        }
        if(min != i){
            std::swap(array[i],array[min]);
        }
    }
}
```

The best case is when the array is already sorted.

- Based on the code, even if the array is already sorted, we still have to do a double pass.
- We can't remediate this with a swap counter.

$$T(n) = O(n)O(n) = O(n^2)$$

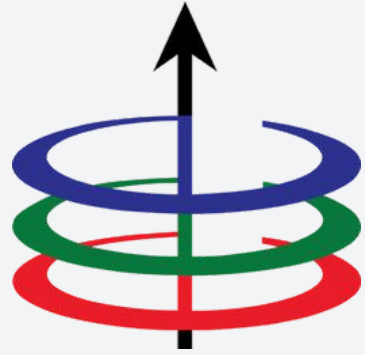
Sorting Algorithms Wrapped 2025

Algorithm	Proved?	Time Complexity	
		Best Case	Worst Case
Insertion Sort	Yes!	$O(n)$	$O(n^2)$
Bubble Sort	Yes!	$O(n)$	$O(n^2)$
Selection Sort	Assignment :)	$O(n^2)$	$O(n^2)$
Merge Sort	Next Week		
Quick Sort			

Any questions?

Quiz ulit! Joke pahinga tayo :)

Just do your assignment in proving selection sort. Submit tomorrow by 11:59PM.



Lecture 2.3

Sorting Algorithms I

EEE 121 THVW1

Steven S. Sison